YOLO Object Detection

Structured Notes

YOLOv1 Architecture, Bounding

Boxes, IoU, Non-Max Suppression,
Mean Average Precision, and YOLO Loss Components

Contents

1 Introduction to YOLO **2**
1.1 Object Classification vs Object Detection . . . 2
1.2 Confidence Score . . . 2

2 Bounding Box Representation **2**
2.1 Vector Size Calculation . . . 2

3 Why YOLO Over Traditional Algorithms **3**

4 Grid-Based Detection in YOLO **3**
4.1 Grid Cell Size Calculation . . . 3
4.2 Total Bounding Boxes . . . 3

5 Training Data Preparation **4**
5.1 Annotation Files . . . 4
5.2 Creating Annotations with MakeSense.ai . . . 4
5.3 Handling Image Transformations . . . 4

6 YOLOv1 Architecture **5**
6.1 Architecture Configuration . . . 5
6.2 CNN Block . . . 5
6.3 Layer-by-Layer Architecture . . . 5
6.4 Repeated Blocks (List Configuration) . . . 5
6.5 Fully Connected Layers . . . 6

7 Library Imports for YOLOv1 Implementation **6**
7.1 Random Seed Initialization . . . 6

8 Intersection over Union (IoU) **6**
8.1 Coordinate Conversion to YOLO Format . . . 7

9 Non-Max Suppression (NMS) **7**

10 Mean Average Precision (mAP) **8**
10.1 Computing mAP . . . 8
10.2 Plotting Predictions . . . 8

11 YOLO Loss Function **8**
11.1 Box Coordinate Loss . . . 8
11.2 Object Loss . . . 8
11.3 No Object Loss . . . 9
11.4 Class Loss . . . 9

12 Dataset Class and Data Loading **9**
12.1 Custom Dataset Class . . . 9
12.2 Data Loading . . . 9
12.3 Saving Model Checkpoints . . . 9

13 Summary **9**

1 Introduction to YOLO

YOLO — You Only Look Once

YOLO stands for **You Only Look Once**. It is a real-time object detection algorithm that detects what objects are present in an image and where they are located, all in a single forward pass through the network.

1.1 Object Classification vs Object Detection

Image classification algorithms take an image as input and output a single label indicating the class of the object present. For example, given an image containing a dog and a cat, a classification algorithm outputs only whether the image contains a dog or a cat. It provides **no information about the location** of the object within the image.

Object detection algorithms like YOLO perform two tasks simultaneously:

1. **Classification**: Identifying *what* the object is (e.g., dog, cat, apple).
2. **Localization**: Identifying *where* the object is located in the image by drawing a **bounding box** (a rectangular box) around the detected object.

Key Distinction

Classification tells you *what* is in the image. Object detection tells you *what* is in the image **and** *where* it is located, by drawing bounding boxes around each detected object.

1.2 Confidence Score

Each bounding box produced by YOLO is accompanied by a **confidence score**. This score indicates how certain the algorithm is that a particular object is present within the bounding box. For example, a confidence score of 0.95 on an apple detection means the model is 95% sure the detected object is an apple.

2 Bounding Box Representation

Bounding Box Vector

A **bounding box** in YOLO is represented as a vector containing specific values that encode both the location and classification of a detected object. For a system with C classes, the bounding box vector has size $5 + C$.

The bounding box vector contains the following components:

Bounding Box Vector $= [P,\ B_x,\ B_y,\ B_w,\ B_h,\ C_1,\ C_2,\ \ldots,\ C_n]$

- P — **Probability** that an object is present in the cell. $P = 1$ indicates an object is present; $P = 0$ indicates no object.
- B_x, B_y — **Center coordinates** of the bounding box relative to the grid cell.
- B_w, B_h — **Width and height** of the bounding box relative to the full image.
- $C_1, C_2, \ldots, C_n$ — **Class probabilities**. For a two-class problem (e.g., dog and cat): if the object is a dog, then $C_1 = 1, C_2 = 0$; if a cat, then $C_1 = 0, C_2 = 1$.

2.1 Vector Size Calculation

The size of the bounding box vector depends on the number of classes:

Number of Classes	Vector Components	Vector Size
2 classes	$P + B_x + B_y + B_w + B_h + C_1 + C_2$	7
4 classes	$P + B_x + B_y + B_w + B_h + C_1 + C_2 + C_3 + C_4$	9
n classes	$5 + n$	$5 + n$

3 Why YOLO Over Traditional Algorithms

Before YOLO, traditional object detection algorithms such as R-CNN and its variants existed. YOLO was developed to address the following limitations of these algorithms:

- Traditional algorithms like R-CNN **look at the image multiple times** (multiple passes), making them slow.
- They are **computationally expensive**.
- YOLO looks at the image **only once** (hence the name) and makes predictions in a single forward pass, making it **fast and efficient**.

YOLOv1 Limitations

While YOLOv1 is fast and efficient, it has known limitations:

- Poor at detecting **small objects**
- Bounding boxes may not be very accurate
- Can miss objects when they are **clustered together**
- Struggles with **multiple objects** in close proximity

These limitations led to the development of subsequent YOLO versions (v2, v3, v4, etc.).

4 Grid-Based Detection in YOLO

Grid Division

YOLO divides the input image into an $S \times S$ grid of cells. For each cell, the model predicts a bounding box vector. The grid size can be any value, for example 7×7 or 4×4.

4.1 Grid Cell Size Calculation

If the input image has size $N \times N$ pixels and is divided into an $S \times S$ grid, then the size of each grid cell is:

$$\text{Cell Size} = \frac{N}{S} \times \frac{N}{S}$$

Example: For a 16×16 image divided into a 2×2 grid:

- Number of grid cells $= 2 \times 2 = 4$
- Size of each cell $= \frac{16}{2} \times \frac{16}{2} = 8 \times 8$ pixels

4.2 Total Bounding Boxes

For a 7×7 grid, the total number of bounding box predictions is $7 \times 7 = 49$. Each cell produces one bounding box vector. If the vector size is 7 (for a 2-class problem), then there are 49 bounding box vectors, each of size 7.

After prediction, YOLO combines all grid cell predictions to produce the final detection output for the entire image.

5 Training Data Preparation

5.1 Annotation Files

Neural networks understand only numerical data. For each training image, the model requires a corresponding **label file** (in XML or text format) that contains bounding box information for all objects in that image.

Annotation Labels

For each image in the training dataset, there is a corresponding **XML or text file** containing the bounding box coordinates and class labels for every object present in that image. This acts as the ground truth (target label) for supervised learning.

The label file contains:

- The probability of object presence (P)
- Center coordinates of the bounding box (B_x, B_y)
- Width and height of the bounding box (B_w, B_h)
- Class labels ($C_1, C_2, \ldots$)

Input Size Constraint

The size of all input images must be **constant** (same dimensions) for the neural network. However, the **bounding box sizes can vary** from image to image.

5.2 Creating Annotations with MakeSense.ai

The tool **MakeSense.ai** is used to create bounding box annotations for training data:

1. Open MakeSense.ai and click "Get Started"
2. Drop all training images into the interface
3. Select "Object Detection" as the task type
4. Create a label list (e.g., "fish", "apple", "orange")
5. For each image, draw bounding boxes around objects of interest:
   - Be **very precise** — fit the bounding box tightly around the object
   - Avoid capturing areas beyond the object (excess background area reduces training quality)
   - For images with **multiple objects**, draw separate bounding boxes around each object and assign the appropriate label
6. Export annotations via Actions → Export Annotations
7. Export in **VOC XML format** for use with YOLO

Bounding Box Precision

When drawing bounding boxes, ensure they are **confined tightly** around the target object. Drawing a bounding box that captures too much background area introduces noise and degrades model performance.

5.3 Handling Image Transformations

When input images are resized (e.g., for uniform input dimensions), the bounding box annotations are **automatically transformed** accordingly. If the input image is resized, the bounding box coordinates scale proportionally.

6 YOLOv1 Architecture

6.1 Architecture Configuration

The YOLOv1 architecture is defined as a **blueprint** — a predefined configuration specifying the types and parameters of each layer. This configuration uses three data structures:

Architecture Config Format

The architecture is specified using three types of entries:

1. **Tuple**: (K, F, S, P) — defines a convolution layer with kernel size K, number of filters F, stride S, and padding P
2. **String "M"**: defines a max pooling layer with kernel size 2 and stride 2
3. **List**: $[\text{tuple}_1, \text{tuple}_2, \text{repeat}]$ — defines a block of two convolution layers repeated a specified number of times

6.2 CNN Block

Each tuple in the architecture config triggers the execution of a **CNN Block** that performs three operations in sequence:

1. **Convolution** (Conv2D) with specified kernel size, filters, stride, and padding
2. **Batch Normalization**
3. **Leaky ReLU** activation

6.3 Layer-by-Layer Architecture

The complete YOLOv1 architecture consists of the following layers:

Layer	Type	Configuration
1	Conv + BN + LeakyReLU	Kernel: 7×7, Filters: 64, Stride: 2, Padding: 3
2	Max Pooling	Kernel: 2×2, Stride: 2
3	Conv + BN + LeakyReLU	Kernel: 3×3, Filters: 192, Stride: 1, Padding: 1
4	Max Pooling	Kernel: 2×2, Stride: 2
5	Conv + BN + LeakyReLU	Kernel: 1×1, Filters: 128, Stride: 1, Padding: 0
6	Conv + BN + LeakyReLU	Kernel: 3×3, Filters: 256, Stride: 1, Padding: 1
7	Conv + BN + LeakyReLU	Kernel: 1×1, Filters: 256, Stride: 1, Padding: 0
8	Conv + BN + LeakyReLU	Kernel: 3×3, Filters: 512, Stride: 1, Padding: 1
9	Max Pooling	Kernel: 2×2, Stride: 2
10–17	Repeated Block ($\times 4$)	1×1, 256 filters $\rightarrow$ 3×3, 512 filters
18	Conv + BN + LeakyReLU	Kernel: 1×1, Filters: 512, Stride: 1, Padding: 0
19	Conv + BN + LeakyReLU	Kernel: 3×3, Filters: 1024, Stride: 1, Padding: 1
20	Max Pooling	Kernel: 2×2, Stride: 2
21–24	Repeated Block ($\times 2$)	1×1, 512 filters $\rightarrow$ 3×3, 1024 filters
25	Conv + BN + LeakyReLU	Additional convolution layers
26	Flatten	Flatten output to 1D
27	Fully Connected	Linear layer with 1024 neurons
28	Dropout + LeakyReLU	Regularization and activation
29	Fully Connected	Linear layer with 496 neurons
30	Fully Connected	Output layer (grid $\times$ grid $\times$ ($5 \times$ B + C))

6.4 Repeated Blocks (List Configuration)

When the architecture config contains a list, it defines a block of two convolution layers that is repeated multiple times. For example:

List entry: $[(1, 256, 1, 0),\ (3, 512, 1, 1),\ 4]$

This means:

1. First convolution: 1×1 kernel, 256 filters, stride 1, padding 0
2. Second convolution: 3×3 kernel, 512 filters, stride 1, padding 1
3. This pair of convolutions is repeated **4 times** in total

6.5 Fully Connected Layers

After the convolution and pooling layers, the output is processed through fully connected layers:

1. **Flatten** the output of the last convolutional layer into a 1D vector
2. Connect to a **Linear layer** with 1024 neurons
3. Connect to a **Linear layer** with 496 neurons
4. Apply **Dropout** for regularization
5. Apply **Leaky ReLU** activation
6. Connect to the **output Linear layer**

7 Library Imports for YOLOv1 Implementation

The following libraries are required for implementing YOLOv1:

YOLOv1 Library Imports

```
import torch                        # PyTorch framework, GPU support
import torch.nn as nn               # Neural network modules, layers, losses
import pandas as pd                 # Data handling, tables
import os                           # File and directory operations
import PIL                          # Image processing
from skimage import io              # Read/write operations on images
import numpy as np                  # Numerical computations on arrays
from PIL import Image               # Load and manipulate images
import matplotlib.pyplot as plt     # Plot images and graphs
from matplotlib import patches      # Draw bounding boxes
from torchvision import transforms  # Image transformations (resize, etc.)
from torch import optim             # Optimizers (Adam, SGD)
import torchvision.transforms.functional as FT  # Low-level image ops
from torch.utils.data import DataLoader  # Create batches, load data
from tqdm import tqdm               # Progress bar for epochs
from collections import Counter     # Count frequencies of elements
import cv2                          # Image processing (OpenCV)
import xml.etree.ElementTree as ET  # Parse XML annotation files
```

Listing 1: Required Libraries for YOLOv1

7.1 Random Seed Initialization

A random seed (e.g., `seed = 123`) is set to ensure **reproducibility**. Without fixing the seed, each execution of the code would initialize weights with different random values, leading to slightly different outputs. Fixing the seed ensures the same weight initialization every time.

8 Intersection over Union (IoU)

Intersection over Union

Intersection over Union (IoU) is a metric that measures the overlap between a predicted bounding

box and the ground truth bounding box. It is used to evaluate how accurately the model predicts object locations.

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

- IoU ranges from **0 to 1**
- IoU = 0: No overlap (no match)
- IoU = 1: Perfect overlap (perfect match)
- The **higher** the IoU, the better the prediction

8.1 Coordinate Conversion to YOLO Format

Before computing IoU, bounding box coordinates must be converted from $(X_{\min}, Y_{\min}, X_{\max}, Y_{\max})$ to the YOLO format of (top, left, right, bottom). The conversion formulas are:

$$\text{left} = \frac{X_1 - X_2}{2}$$
$$\text{top} = Y_1 - \frac{H}{2}$$
$$\text{right} = X_2 + \frac{W}{2}$$

This conversion is applied to both **predicted** and **ground truth** bounding boxes before computing the IoU.

9 Non-Max Suppression (NMS)

Non-Max Suppression

Non-Max Suppression (NMS) is a post-processing technique used to eliminate duplicate bounding box predictions for the same object. When the model predicts multiple overlapping bounding boxes for a single object, NMS keeps only the one with the **highest IoU** (or highest confidence score) and removes the rest.

The NMS algorithm works as follows:

1. For each object class, sort all predicted bounding boxes by their confidence scores in descending order
2. Select the bounding box with the highest confidence score
3. Compute the IoU between this box and all remaining boxes
4. Remove all boxes whose IoU with the selected box exceeds a threshold (they are considered duplicates)
5. Repeat until no boxes remain

Why NMS is Necessary

It is possible for the neural network to predict multiple bounding boxes for a single object. NMS ensures that only the **best** bounding box (with the highest confidence) is retained for each detected object, removing all duplicate detections.

10 Mean Average Precision (mAP)

Mean Average Precision

Mean Average Precision (mAP) is the primary performance metric for object detection models. It evaluates how well the model detects and localizes objects across all classes. The higher the mAP, the better the model's performance.

10.1 Computing mAP

The mAP computation involves the following steps:

1. For each class, compare **predicted bounding boxes** with **ground truth bounding boxes**
2. Compute **Precision** and **Recall** at different IoU thresholds
3. Plot the **Precision-Recall curve** for each class
4. Compute the **Area Under the Precision-Recall Curve (AP)** for each class
5. Average the AP values across all classes to get the **mAP**

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} \text{AP}_i$$

Where N is the number of classes and AP_i is the Average Precision for class i.

10.2 Plotting Predictions

To visualize detections, the grid-level predictions must be converted back to image-level coordinates. The `cellboxes_to_boxes` function converts bounding box predictions from the grid format (relative to each cell) to absolute coordinates on the full image, enabling the bounding boxes to be plotted on the original image.

11 YOLO Loss Function

The YOLOv1 loss function comprises **four components**, each computed using Mean Squared Error (MSE):

YOLO Loss Components

1. **Box Coordinate Loss**: MSE between predicted and ground truth bounding box coordinates (B_x, B_y, B_w, B_h)
2. **Object Loss**: MSE between the predicted objectness score and 1 (when an object is present)
3. **No Object Loss**: MSE between the predicted objectness score and 0 (when no object is present)
4. **Class Loss**: MSE between the predicted class probabilities and the ground truth one-hot encoded class vector

11.1 Box Coordinate Loss

This measures the error in predicting the bounding box location and size. It computes the MSE between the predicted coordinates (B_x, B_y, B_w, B_h) and the ground truth coordinates.

11.2 Object Loss

For cells where an object is present, this loss measures how confident the model is in detecting the object. If the ground truth says an object is present (score = 1) and the model predicts a confidence of 0.6, the object loss is the MSE between 0.6 and 1.

11.3 No Object Loss

For cells where no object is present, this loss penalizes false positive predictions. If the model predicts 0.6 confidence for a cell with no object (ground truth = 0), the no object loss is the MSE between 0.6 and 0.

11.4 Class Loss

This measures the error in classifying detected objects. For example, if an image contains an orange with a one-hot encoding of $[0, 1, 0]$ and the model predicts $[0.6, 0.9, 0.3]$, the class loss is the MSE between these two vectors.

> **Total YOLO Loss** $= \lambda_{\text{coord}} \cdot L_{\text{box}} + L_{\text{obj}} + \lambda_{\text{noobj}} \cdot L_{\text{noobj}} + L_{\text{class}}$
>
> Where λ_{coord} and λ_{noobj} are weighting factors that balance the contribution of each loss component.

12 Dataset Class and Data Loading

12.1 Custom Dataset Class

For the fruit detection task (apple, banana, orange), a custom PyTorch dataset class (`FruitImageDataset`) is created that:

1. Defines the classes: apple, banana, and orange
2. Specifies the path to the dataset
3. Defines the image size (all images must be resized to the same dimensions)
4. Reads each image and its corresponding XML annotation file
5. Extracts bounding box coordinates from the XML file
6. Converts the bounding box coordinates into YOLO format

12.2 Data Loading

The data loading process uses `pd.Series` to create a mapping between each image file and its corresponding XML annotation file. This is done for both training and testing datasets, ensuring that each image has its ground truth bounding box information properly associated.

The `DataLoader` from PyTorch is used to:

- Create **batches** from the dataset
- Load data efficiently for training
- Handle shuffling and parallel data loading

12.3 Saving Model Checkpoints

During training, model checkpoints are saved periodically. This allows the trained model weights to be restored later without retraining from scratch.

13 Summary

- **YOLO** (You Only Look Once) is a real-time object detection algorithm that performs classification and localization simultaneously in a single forward pass
- A **bounding box vector** contains: probability P, center coordinates (B_x, B_y), dimensions (B_w, B_h), and class probabilities $(C_1, \ldots, C_n)$; vector size is $5 + n$ for n classes
- YOLO divides the image into an $S \times S$ **grid**; each cell predicts a bounding box vector, producing S^2 total predictions
- Training data requires **annotation files** (XML/text) for each image, created using tools like MakeSense.ai

- The **YOLOv1 architecture** consists of convolution layers (with batch normalization and Leaky ReLU), max pooling layers, and fully connected layers
- **Intersection over Union (IoU)** measures the overlap between predicted and ground truth bounding boxes (0 = no match, 1 = perfect match)
- **Non-Max Suppression (NMS)** removes duplicate bounding box predictions, keeping only the one with the highest confidence for each object
- **Mean Average Precision (mAP)** is the overall performance metric, computed by averaging the area under the precision-recall curve across all classes
- The **YOLO loss function** has four components: box coordinate loss, object loss, no object loss, and class loss — all computed using MSE
- YOLOv1 limitations include poor small-object detection and struggles with clustered objects, which led to subsequent YOLO versions